Engenharia de Dados (Pandas I e II)

Estruturas, Limpeza e Tratamento para Machine Learning

Prof. João Vítor Rodrigues de Vasconcelos

Engenharia de Computação / Análise e Desenvolvimento de Sistemas
joaovasconcelos@univicosa.com.br

ADS308

Objetivos da Aula

O Caminho do Engenheiro de Dados

Nesta aula, vamos cobrir o ciclo fundamental de preparação de dados, essencial para qualquer aplicação de IA.

1. Pandas I: Estruturas e Manipulação Básica
2. Pandas II: Limpeza e Tratamento de Dados
3. Conclusão e Próximos Passos

Roteiro da Aula

1. Pandas I: Estruturas e Manipulação Básica
2. Pandas II: Limpeza e Tratamento de Dados
3. Conclusão e Próximos Passos

O Ecossistema de Dados

Onde estamos no Pipeline?

Antes de aplicar um algoritmo de Machine Learning (como uma Rede Neural), os dados passam por uma jornada:

1. **Coleta:** Sensores, APIs, Bancos de Dados.
2. **Ingestão:** Trazer para a memória (Pandas entra aqui).
3. **Limpeza:** Tratar erros e ausências (Pandas).
4. **Transformação:** Normalizar e Padronizar (Pandas/Scikit-Learn).
5. **Modelagem:** Treinamento do modelo.

A Biblioteca Pandas

- **Definição:** Biblioteca open-source construída sobre o *NumPy*, oferecendo estruturas de dados de alto desempenho e fáceis de usar.
- **Por que usar?**
  - Substitui o Excel para grandes volumes de dados.
  - Permite automação via scripts Python.
  - Integração nativa com Matplotlib (gráficos) e Scikit-Learn (ML).

Instalação Padrão

```
pip install pandas
```

Estrutura 1: Series

Array Unidimensional

Uma `Series` é como uma coluna de uma tabela ou um vetor. Ela possui dados e um índice (labels).

```
import pandas as pd

# Criando uma Series a partir de uma lista
notas = pd.Series([7.5, 8.0, 9.5], index=["Aluno A", "Aluno B", "Aluno C"])

print(notas)
# Saída:
# Aluno A    7.5
# Aluno B    8.0
# Aluno C    9.5
# dtype: float64
```

Estrutura 2: DataFrame

Tabela Bidimensional

O `DataFrame` é a estrutura principal. Pense nele como uma planilha do Excel ou uma tabela SQL em memória.

- É um conjunto de `Series` que compartilham o mesmo índice (linhas).
- Possui linhas (index) e colunas (columns).
- Permite tipos de dados heterogêneos (uma coluna inteira, outra string, outra float).

Criando DataFrames na Prática

```
dados = {
    "Nome": ["Ana", "Bruno", "Carlos"],
    "Idade": [23, 35, 29],
    "Cidade": ["Viçosa", "Ubá", "Ponte Nova"]
}

# Convertendo dicionário para DataFrame
df = pd.DataFrame(dados)

print(df)
# Saída visual tabular automática no Jupyter/Console
```

Conhecendo seus Dados

Antes de manipular, precisamos "olhar"para os dados.

- `df.head(n)`: Mostra as *n* primeiras linhas.
- `df.tail(n)`: Mostra as *n* últimas linhas.
- `df.info()`: Resumo técnico (tipos de dados, memória usada, não-nulos).
- `df.describe()`: Resumo estatístico (média, desvio padrão, min, max) das colunas numéricas.

Acessando Colunas

Podemos tratar colunas como chaves de dicionário ou atributos (se o nome não tiver espaços).

```
# Retorna uma Series
coluna_idade = df["Idade"]

# Retorna um novo DataFrame (note os colchetes duplos)
sub_tabela = df[["Nome", "Cidade"]]

# Acesso via atributo (cuidado com nomes compostos)
idades = df.Idade
```

Indexação Baseada em Rótulos (.loc)

O `.loc` foca no **NOME** dos índices e colunas.

Sintaxe

```
df.loc[linhas, colunas]
```

```
# Supondo que o índice sejam os nomes
df.set_index("Nome", inplace=True)

# Seleciona linha "Ana" e coluna "Cidade"
cidade_ana = df.loc["Ana", "Cidade"]

# Fatiamento INCLUSIVO (diferente do Python padrão)
# Pega de "Ana" até "Carlos"
fatia = df.loc["Ana":"Carlos", "Idade"]
```

Indexação Baseada em Posição (.iloc)

O `.iloc` foca na **POSIÇÃO NUMÉRICA** (Integer Location).

Sintaxe

```
df.iloc[índice_linha, índice_coluna]
```

```
# Seleciona primeira linha (0) e segunda coluna (1)
dado = df.iloc[0, 1]

# Fatiamento EXCLUSIVO no final (padrão Python)
# Linhas 0 e 1 (exclui a 2)
primeiras_linhas = df.iloc[0:2, :]
```

Filtrando Dados (A "Cláusula WHERE")

Criamos uma máscara booleana para filtrar linhas baseadas em valores.

```
# 1. Criar a condição
mask = df["Idade"] > 25

# 2. Aplicar a máscara
maiores_25 = df[mask]

# Forma direta (mais comum)
filtros = df[df["Cidade"] == "Viçosa"]
```

Operadores Lógicos no Pandas

Em Python puro usamos `and`, `or`, `not`. No Pandas (NumPy) usamos bitwise operators:

- **&** (E / AND)
- | (OU / OR)
- ~ (NÃO / NOT)

Atenção: É obrigatório usar parênteses para separar as condições.

```
# Idade entre 20 e 30 anos
jovens = df[(df["Idade"] >= 20) & (df["Idade"] <= 30)]
```

Roteiro da Aula

1. Pandas I: Estruturas e Manipulação Básica
2. Pandas II: Limpeza e Tratamento de Dados
3. Conclusão e Próximos Passos

Garbage In, Garbage Out

Realidade do Mercado

Dados do mundo real são sujos, incompletos e ruidosos.

- **NaNs:** Falha de sensor, campo opcional em formulário.
- **Outliers:** Erro de digitação, fraude, anomalia sistêmica.
- **Escala:** Unidades diferentes (metros vs milímetros, salário vs idade).

Se você alimentar um modelo de Machine Learning com dados sujos, ele aprenderá padrões errados.

Identificando NaNs

`NaN` = Not a Number (padrão IEEE 754 para ponto flutuante).

```
import numpy as np

# Inserindo um erro proposital
df.iloc[0, 1] = np.nan

# Verificando onde há nulos (retorna True/False)
print(df.isnull())

# Contando nulos por coluna (Essencial!)
print(df.isnull().sum())
```

Estratégia 1: Remoção (Drop)

Se a quantidade de dados faltantes é pequena ($< 5\%$) ou a linha é inútil sem aquele dado.

```
# Remove linhas que tenham pelo menos 1 NaN
df_limpo = df.dropna()

# Remove colunas que tenham nulos (cuidado!)
df_sem_colunas_ruins = df.dropna(axis=1)

# Remove apenas se TODAS as colunas forem NaN
df.dropna(how="all")
```

Estratégia 2: Imputação (Fill)

Substituir o vazio por um valor constante ou lógico.

```
# Preencher com Zero
df.fillna(0, inplace=True)

# Preencher com "Desconhecido" (para categóricos)
df["Cidade"].fillna("Desconhecido", inplace=True)

# Preencher com o valor anterior (Forward Fill) - bom para séries temporais
df.fillna(method="ffill")
```

Estratégia 3: Média/Mediana

Para manter a distribuição estatística dos dados numéricos.

```
# Calculando a média da idade
media_idade = df["Idade"].mean()

# Preenchendo NaNs com a média
df["Idade"].fillna(media_idade, inplace=True)
```

Dica de Ouro

Use a **Mediana** se houver muitos outliers, pois a média é sensível a valores extremos.

O que são Outliers?

São observações que se diferenciam drasticamente das demais.

- **Causa Natural:** Um bilionário numa lista de salários médios.
- **Erro de Dados:** Alguém com 200 anos de idade ou altura negativa.

Impacto

Algoritmos como Regressão Linear e K-Means são "puxados"pelos outliers, distorcendo o resultado final.

Detection: Intervalo Interquartil (IQR)

Uma técnica robusta estatisticamente para definir o que é "normal".

- $Q1$: Quartil 25%
- $Q3$: Quartil 75%
- $IQR = Q3 - Q1$

Regra do Limite:

$$Limite_{inferior} = Q1 - 1.5 \times IQR$$

$$Limite_{superior} = Q3 + 1.5 \times IQR$$

Tudo fora desses limites é considerado Outlier.

Filtrando Outliers na Prática

```
Q1 = df["Salario"].quantile(0.25)
Q3 = df["Salario"].quantile(0.75)
IQR = Q3 - Q1

limite_inf = Q1 - 1.5 * IQR
limite_sup = Q3 + 1.5 * IQR

# Mantendo apenas o que está DENTRO dos limites
df_sem_outliers = df[
    (df["Salario"] >= limite_inf) &
    (df["Salario"] <= limite_sup)
]
```

Por que escalar dados?

Imagine duas variáveis num dataset de imóveis:

1. **Número de Quartos:** 1 a 5.
2. **Preço:** 200.000 a 1.000.000.

Algoritmos baseados em distância (KNN, SVM) e baseados em gradiente (Redes Neurais) vão dar peso total ao "Preço"e ignorar "Quartos", pois os números são maiores. Precisamos colocar tudo na mesma régua.

Normalização (Min-Max Scaling)

Transforma os dados para um intervalo fixo, geralmente entre **0 e 1**.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- **Vantagem:** Mantém a distribuição original exata.
- **Desvantagem:** Muito sensível a outliers (um outlier "esmaga"os outros dados num canto pequeno do intervalo).
- **Uso:** Processamento de Imagens (pixel 0-255 viram 0-1), Redes Neurais.

Aplicando MinMaxScaler

Usamos a biblioteca `scikit-learn` para isso.

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

# Atenção: fit_transform espera uma matriz 2D (use colchetes duplos)
df["Salario_Norm"] = scaler.fit_transform(df[["Salario"]])

print(df["Salario_Norm"].min()) # Será 0.0
print(df["Salario_Norm"].max()) # Será 1.0
```

Padronização (Standardization)

Transforma os dados para terem **Média = 0** e **Desvio Padrão = 1**.

$$Z = \frac{X - \mu}{\sigma}$$

- **Vantagem:** Lida muito melhor com outliers. Não impõe limites fixos (pode ir de -3 a +3 ou mais).
- **Uso:** Algoritmos que assumem distribuição normal (Regressão Logística, Linear Discriminant Analysis).

Aplicando StandardScaler

```
from sklearn.preprocessing import StandardScaler

std_scaler = StandardScaler()

df["Salario_Std"] = std_scaler.fit_transform(df[["Salario"]])

# A média será muito próxima de 0
print(df["Salario_Std"].mean())
```

Comparativo Visual (Conceitual)

Suponha os dados: `[10, 20, 30, 1000 (outlier)]`

MinMax (0 a 1)

O 10 vira 0.0
O 1000 vira 1.0
O 30 vira **0.02**
(Dados "esmagados")

StandardScaler

Centraliza na média.
Mantém a distância relativa em termos de desvios padrões.
O 1000 ficará longe, mas 10, 20 e 30 ainda serão distinguíveis.

Resumo: Qual técnica usar?

Cenário	**Recomendação**	**Por quê?**
Redes Neurais / Imagens	Normalização	Convergência rápida (0 a 1)
Algoritmos de Distância (KNN)	Ambos funcionam	Testar qual performa melhor
Muitos Outliers	Padronização	Mais robusto estatisticamente
Não sei a distribuição	Padronização	Aposta mais segura no geral

Roteiro da Aula

1. Pandas I: Estruturas e Manipulação Básica
2. Pandas II: Limpeza e Tratamento de Dados
3. Conclusão e Próximos Passos

Conexão com o Mercado de Trabalho

A Realidade da Profissão

Um Cientista de Dados Júnior gasta 80% do tempo fazendo exatamente o que vimos hoje: limpando e organizando dados (Data Wrangling).

- Empresas não têm dados "prontos". Eles vêm de planilhas bagunçadas, sistemas legados e APIs instáveis.
- A habilidade de transformar um CSV sujo em um DataFrame limpo e padronizado é o filtro número 1 em entrevistas técnicas.

Próximos Passos e Dúvidas

Dúvidas?

joaovasconcelos@univicosa.com.br